

MODULE 6 · SCHEDULING, RESOURCES & SCALING

Jobs & CronJobs

Run-to-completion workloads, once or on a schedule

Not everything should run forever

Deployments and DaemonSets exist to keep Pods **running indefinitely**. But some work has a natural end: a database migration, a report, a batch import, a nightly cleanup.

A **Job** runs Pods until a task finishes, then stops. A **CronJob** creates a new Job on a schedule. Together they cover every "run this, then be done" use case.

BY THE END YOU CAN

- Explain completions vs parallelism
- Pick a restartPolicy that's valid in a Job
- Read a cron schedule and concurrencyPolicy
- Say when to use a Job vs a CronJob

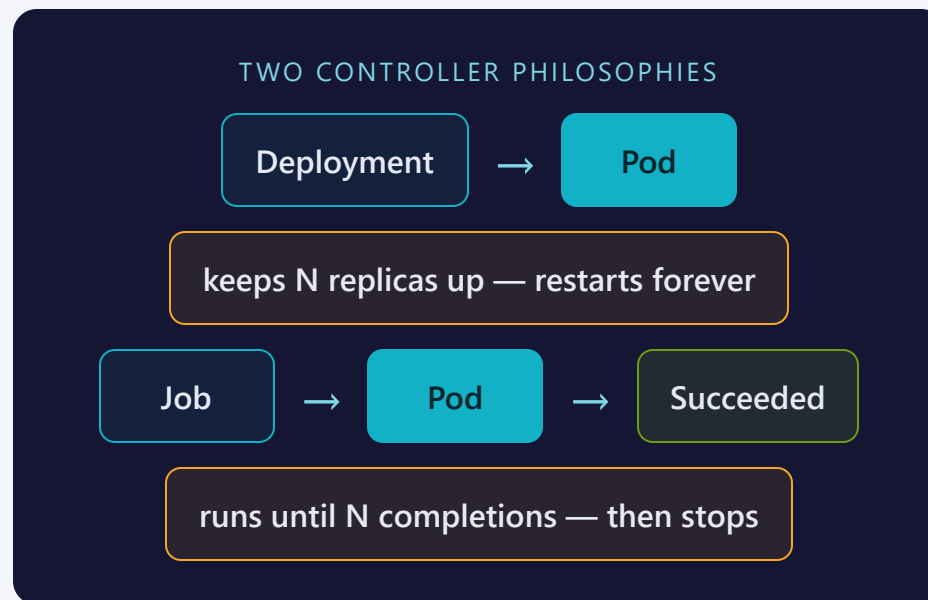
CONCEPT

Run-to-completion, not keep-alive

A Deployment's controller loop never stops: if a Pod exits, it's replaced, forever.

A **Job**'s controller loop has a finish line: it creates Pods until a target number **succeed**, then it stops creating more. The Job object itself stays around recording the result.

A CronJob doesn't run Pods directly — it creates **Jobs**, which then create Pods, exactly as above.



Four knobs decide how a Job runs

FIELD	CONTROLS
<code>completions</code>	Total successful Pods needed to call the Job done
<code>parallelism</code>	How many Pods run at the same time
<code>backoffLimit</code>	Retries allowed before the Job gives up (Failed)
<code>restartPolicy</code>	<code>Never</code> or <code>OnFailure</code> — <code>Always</code> is invalid in a Job



A Job wraps a Pod template

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
  namespace: training
spec:
  completions: 3
  parallelism: 2
  template:
    metadata:
      labels:
        app: hello-job
    spec:
      containers:
        - name: worker
          image: busybox:1.36
          command:
            - /bin/sh
            - -c
            - 'echo "Hello from Job" && sleep 5'
          restartPolicy: Never
      backoffLimit: 4
```

Imperative to explore, declarative to keep

IMPERATIVE — FAST, ONE-OFF

```
kubectl create job hello --image=busybox:1.36 -- echo hi
```

Great for a quick one-shot task, no file to manage

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f job.yaml
```

You own the file — completions, parallelism, retries are reviewable

THE BRIDGE TRICK

Generate YAML from an imperative command, then edit & keep it:

```
$ kubectl create job hello --image=busybox:1.36 \  
  --dry-run=client -o yaml -- echo hi > job.yaml
```

SEE IT

A CronJob is a Job factory on a timer

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: date-printer
  namespace: training
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        metadata:
          labels:
            app: date-printer
        spec:
          containers:
            - name: date
              image: busybox:1.36
              command:
                - /bin/sh
                - -c
                - 'echo "Current date: $(date)"'
          restartPolicy: OnFailure
      successfulJobsHistoryLimit: 3
      failedJobsHistoryLimit: 1
```

Every tick, the CronJob controller wraps this jobTemplate in a new, independent **Job** object (name suffixed with a timestamp) — which then runs Pods exactly like a standalone Job. `successfulJobsHistoryLimit` / `failedJobsHistoryLimit` cap how many old Jobs are kept around.

Where each one fits

FIELD	CONTROLS
<code>schedule</code>	Standard cron syntax: <code>min hour day month weekday</code>
<code>concurrencyPolicy</code>	<code>Allow</code> (default) · <code>Forbid</code> · <code>Replace</code>
<code>startingDeadlineSeconds</code>	How late a missed run may still start
<code>suspend</code>	<code>true</code> pauses scheduling without deleting the CronJob

MENTAL MODEL

Job — a one-off batch task you trigger now (migration, backfill, ad-hoc report).

CronJob — the same kind of task, but recurring on a schedule (nightly backup, hourly sync).

Where people trip

- **restartPolicy: Always.** Invalid inside a Job's template — use Never or OnFailure.
- **backoffLimit exhausted.** The Job goes Failed and stays that way — nothing retries it further.
- **Completed Pods pile up.** Job Pods aren't deleted automatically; set `ttlSecondsAfterFinished` to clean up.
- **concurrencyPolicy: Allow by default.** A slow Job can overlap with the next tick and pile up Pods.
- **Deadlines aren't queues.** A run missed past `startingDeadlineSeconds` is skipped, not run late.
- **Deleting a CronJob ≠ deleting its Jobs.** Already-created Jobs and Pods are left behind.

Commands to keep at hand

```
$ kubectl create job NAME --image=IMG -- CMD    # one-off Job
$ kubectl get jobs -n NS                        # completions status
$ kubectl describe job NAME -n NS               # events, backoff
$ kubectl logs -n NS -l app=LABEL               # output from all Pods
$ kubectl create job NAME --from=cronjob/CJ -n NS # trigger manually
$ kubectl patch cronjob CJ -n NS -p '{"spec":{"suspend":true}}' # pause
```

Tip: `kubectl wait --for=condition=complete job/NAME` beats polling in scripts.

RECAP

Jobs & CronJobs in one breath

- Job = run-to-completion; Deployment = keep-alive forever
- completions / parallelism shape how many Pods, how fast
- restartPolicy in a Job is Never or OnFailure only
- CronJob = a Job factory on a cron schedule, guarded by concurrencyPolicy

HANDS-ON

Now run Lab 6.6
[labs/6.6/](#)

Next: [7.1 — Namespaces](#)